HACKATHON

# Report

# DEPTHGUARD 360

Edge- Preserving Depth Map Upsampling and Refinement

| | |
|---|---|
| **Problem ID** | P4 |
| **Problem Title** | Edge- Preserving Depth Map Upsampling and Refinement |
| **Team Name& Team ID** | Brocoders, T29 |
| **Team Members** | B R Nishchala(1RUA24CSE0080), Athreya Ramesh(1RUA24CSE0074), Arnav Deshmukh(1RUA24CSE0066), Ayush (1RUA24CSE0077), Aryaman Misra(1RUA24CSE0069) |
| **GitHub Repository** | https://github.com/cv-hackathon-team/CV-hackathon.git |
| **URL** | https://cv-hackathon-production-25bb.up.railway.app/ |
| **Date** | 7 November 2026 |

**AI assistance declaration.** AI assistance (Claude, by Anthropic) was used for system-design brainstorming, for drafting the design document and the text of this report and for laying out the architecture diagrams. Estimated AI-generated share of this report: [30 %] .

# Contents


1. Abstract
2. Introduction
   2.1 Background
   2.2 Motivation
   2.3 Applications
3. Problem Statement and Objectives
   3.1 Problem Statement
   3.2 Objectives
4. System Design and Implementation
   4.1 Overall System Architecture
   4.2 Classical Upsampling Methods
   4.3 Adaptive Method Selection
   4.4 Hybrid Reconstruction Approach
   4.4.1 Fusion Strategy
   4.4.2 Selection and Decision Process
   4.5 Intelligent Processing Modules
   4.5.1 Depth Rescue
   4.5.2 DepthGuard
   4.5.3 Occlusion-X
   4.5.4 Texture Trap
   4.5.5 Temporal Depth Repair
   4.5.6 Geometry First
   4.6 Implementation and Code Structure
   4.7 Deep Learning Approach
   4.7.1 Model Architecture
   4.7.2 Loss Functions and Training
   4.7.3 Dataset Preparation and Preprocessing
   4.7.4 Training Setup
5. Experimental Results and Analysis
   5.1 Evaluation Metrics
   5.2 Results at Different Upsampling Scales
   5.3 Visual Results and Intermediate Outputs
   5.4 Ablation Study
   5.5 Runtime and Resource Usage
6. Limitations and Failure Cases
   6.1 Observed Failure Cases
   6.2 Causes and Possible Improvements
   6.3 Current Limitations
7. Deployment and Reproducibility
   7.1 Quickstart

7.2 Deployment Format
7.3 Environment and Dependencies
7.4 Reproducibility
8. References

# 1. Abstract

Depth maps give information about how far objects are from a camera. Depth maps are used a lot in computer vision tasks such as navigation, robotics, 3D scene understanding and augmented reality. But depth maps that come from sensors like LiDAR or depth cameras have missing data, sparse areas, noise and wrong object edges. These problems lower how reliable depth-based applications are.

This project introduces DEPTHGUARD 360 a depth map upsampling and refinement system that is made to make low-resolution, sparse and damaged depth maps better. DEPTHGUARD 360 looks at mixing classic computer vision tricks with deep-learning reconstruction to get high-resolution depth maps that keep object edges.

The DEPTHGUARD 360 framework uses confidence-aware reconstruction checks for depth damage works around occlusions compares depth borders can use time-based data and refines geometric consistency. Classic methods like nearest-neighbour, bilinear and bicubic interpolation, bilateral upsampling and guided filtering are tested together with a deep-learning reconstruction model.

The system is tested with numbers such as RMSE, MAE, PSNR, SSIM how well edges are kept and how fast it runs. The experiments compare reconstruction methods, with the hybrid way to see how much better or worse the mix of learned and classic techniques is.

# 2. Introduction

## 2.1 Background Information

Depth estimation is a task in computer vision. It helps us find how far objects in a picture are from the camera. A depth map shows this in an image. Each pixel in that image holds the guessed distance to the scene point it represents.

Depth information is useful for areas. In self-driving cars, it lets the vehicle read the road and spot obstacles. In robots it helps with moving and interacting with things. In reality it helps the system grasp the real world. In reconstruction it lets us build models of objects and scenes.

Getting correct depth data is hard. Depth sensors can give gaps or noisy data. This happens when surfaces reflect light, when light levels change, when the sensor has limits when objects hide one another or when errors occur. Increasing resolution by upsampling can help,. Simple interpolation can blur edges give wrong depth values and smooth out details you do not want.

For example when two objects stand at distances interpolation may mix their depth values near the edge. That makes the change, between foreground and background look wrong.

DEPTHGUARD 360 looks into a mix of reconstruction confidence estimation, boundary preservation and geometric refinement. DEPTHGUARD 360 aims to fix these problems.

## 2.2 Motivation

The motivation is that raising the resolution of a depth map doesn't necessarily increase the accuracy of that depth map.

A reconstruction method must consider whether the original measurements are reliable whether missing parts can be inferred from surrounding areas whether RGB edges align with the real depth boundaries and whether the reconstructed surfaces remain geometrically correct. Traditional interpolation methods generally do not model all of these conditions. Deep learning methods can learn relationships, between an image and its depth. They may produce incorrect results when the input contains strange textures missing data or unfamiliar scenes.

### 2.3 Applications

- Robotic: obstacle understanding.
- Driving: scene perception.
- 3D: scene modelling.
- Mixed reality:
- Industrial: measurement.

## 3. Problem Statement and Objectives

### 3.1 Problem Statement

Depth maps that are low in resolution sparse and full of noise have information in space and might not keep important edges of objects. Traditional methods of making depth maps bigger often lead to edges and wrong surface estimates. Methods that rely on learning might have trouble with uncertainty, the way the data is corrupted and keeping the right shape.

The problem that this project is trying to solve is to create a way to make depth maps bigger and improve them. This method should create high-resolution depth information while making sure the errors are as small as possible. It should also protect the edges of objects. Keep the shape right.

### 3.2 Objectives

- Build ways to enlarge depth maps.
- Explore using learning to build depth maps from color and depth details.
- Find the parts of depth maps that are missing, noisy or unreliable.
- Find out how confident we can be about depth values that are produced when the code permits.
- Keep the edges of objects and the places where depth changes.
- Distinguish edges caused by texture from real depth changes.
- Check if shape remains correct when improving depth maps.
- Use time information from videos when data is available to obtain depth.
- Compare methods, deep learning methods and a mix of both for building depth.
- Check how good the results are, by using numbers and looking at images.

## 4. Technical Architecture & Implementation

The pipeline has five layers: presentation , orchestration, reconstruction (classical, learned, hybrid), data, and evaluation.

## 2.1 Classical Algorithmic Baseline

Classical algorithms create reference points and can also offer candidate estimates for hybrid fusion.

| Algorithm | Role | Expected strengths and limitations |
|---|---|---|
| Nearest-neighbour | Copies the closest measured depth value. | Fast and preserves samples, but produces block-like output. |
| Bilinear | Weighted average of neighbouring depth values. | Smooth output, but may blend different surfaces. |
| Bicubic | Cubic interpolation over a larger neighbourhood. | Smooth reconstruction; may blur boundaries or overshoot. |
| Joint bilateral upsampling | Uses RGB guidance with spatial and range similarity. | Can preserve some edges; may copy texture edges. |
| Guided filtering | Uses a guidance image to refine depth structure. | Efficient edge-aware refinement; depends on guidance and parameters. |

## 2.2 Adaptive Selection Logic

An adaptive implementation can use input statistics such as valid-depth ratio, local depth variance, RGB gradient magnitude and depth-gradient magnitude. A strong RGB edge with supporting depth change may be a texture edge rather, than a physical boundary. A depth discontinuity supported by depth samples is stronger evidence of a real surface boundary. These cues can help decide filtering strength, thresholds or the selection of a reconstruction path.

The exact adaptive rule must be explained based on the source code. If the current implementation uses fixed filter parameters say that instead of describing an adaptive mechanism that does not exist.

## 2.3 Hybrid Integration & Adaptive Mechanism

### Fusion Strategy

The hybrid stage merges an estimate and a learned depth prediction. In the stage the classical branch produces a result such as interpolation or edge-aware smoothing. In the stage the neural branch applies patterns learned from RGB images to predict depth. Confidence levels, input validity and pixel proximity, to a boundary all affect how much each estimate contributes in the stage.

A general way to combine them is with a confidence-weighted formula:

$$D_{\text{fused}}(p) = w(p) \times D_{\text{DL}}(p) + [1 - w(p)] \times D_{\text{classical}}(p)$$

Here D_DL is the depth from the learned model D_classical is the depth estimate and w(p) is a weight for each pixel. This weight could be based on confidence, input validity or the chance that a pixel lies on a boundary. This equation is a guide. The actual fusion rule at the code level might be different. If the system uses filtering step by step or picks one output of blending describe that instead.

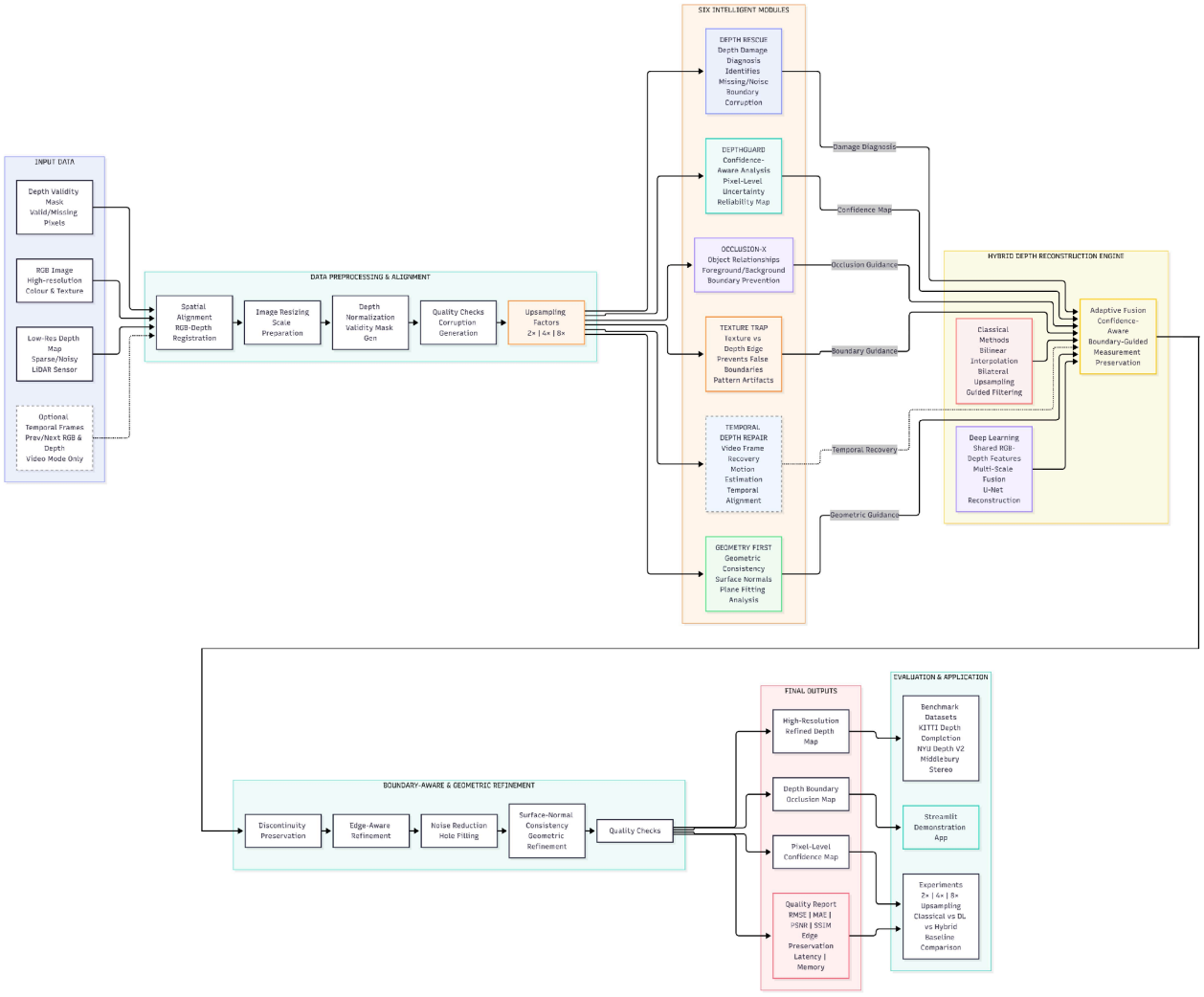
### Decision Logic

The decision process must be aware of both pixels and regions. Where depth measurements are reliable keep them. In areas where depth is missing or uncertain reconstruct it. Near real depth boundaries avoid blending pixels from sides of an object. If there is an edge, in the RGB image but no matching depth edge reduce the influence of color cues to avoid copying textures from one side to another.

Geometric refinement might involve estimating surface normals identifying flat regions or fitting planes using RANSAC. These methods should be local and aware of boundaries. They must not force surfaces into flat planes or smooth across real object edges. The rules must respect scene structure.

| Named module | Responsibility | Evidence required |
|---|---|---|
| DEPTH RESCUE | Diagnoses missing, sparse, noisy or boundary-corrupted depth. | Rules/classifier and diagnostic output. |
| DEPTHGUARD | Estimates reliability and guides reconstruction. | Confidence map, prediction head or implemented weighting. |
| OCCLUSION-X | Protects foreground/background depth relationships. | Boundary, segmentation or depth-order logic. |
| TEXTURE TRAP | Separates RGB texture edges from depth boundaries. | RGB-depth gradient analysis or trained boundary branch. |
| TEMPORAL DEPTH REPAIR | Uses aligned adjacent frames for video reconstruction. | Frame alignment, motion/occlusion handling and temporal fusion. |
| GEOMETRY FIRST | Checks local surface and planar consistency. | Normal estimation, plane fitting or geometric refinement. |

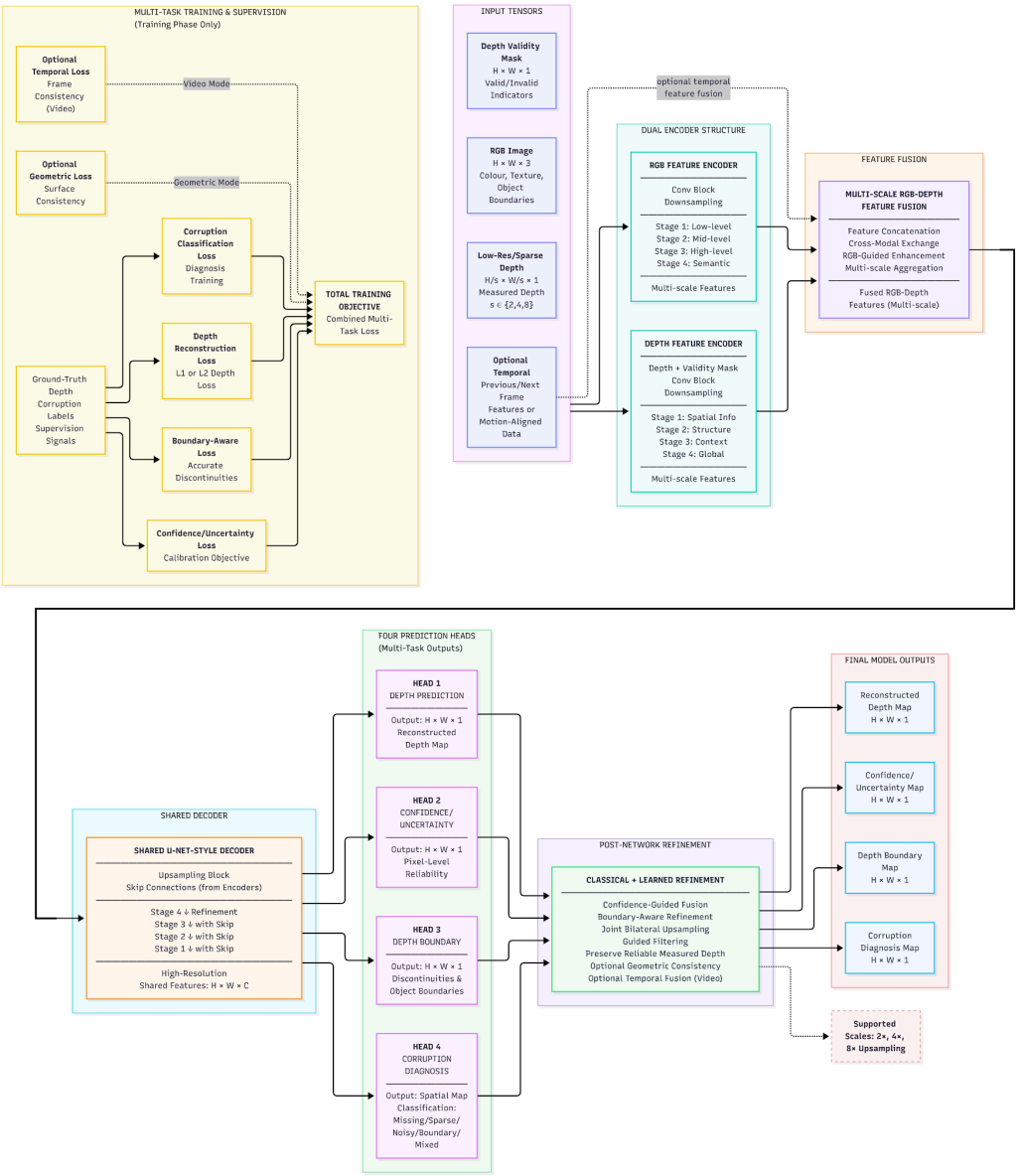
## 2.4 Architecture-to-Code Consistency




## 2.2 Deep Learning Pipeline

**Model Architecture**: The proposed neural subsystem is a shared RGB-D encoder-decoder.The RGB encoder pulls out appearance and image-structure features.The depth encoder works on a low-resolution or sparse depth map and a validity mask. The neural subsystem can tell measured values apart from missing pixels.Multi-scale feature fusion joins the two representations. The U-Net-style decoder restores resolution and uses skip connections to keep spatial details. When the neural subsystem is used the decoder can give prediction heads: a depth head for high-resolution depth a confidence or uncertainty head for reliability a boundary head for likely depth discontinuities and a

corruption head for input-quality classification. These heads are branches of the shared network, not models. Only the heads that are, in the code and trained can be said to be implemented. The architecture is suitable because RGB and depth have information characteristics while skip connections and multi-scale features help keep object boundaries and spatial context.




**Loss functions and training.** A masked combined loss, with invalid ground-truth pixels excluded:

| Loss term | Purpose |
|---|---|
| Depth reconstruction (L1 or L2) | Error against ground-truth depth on valid pixels. |
| Boundary-aware / gradient | Keeps depth discontinuities and local gradients accurate. |
| Confidence / uncertainty | Calibrates the confidence head. |
| Corruption classification | Trains the diagnosis head (missing / sparse / noisy / boundary / mixed). |
| Geometric consistency | Surface consistency between neighbouring pixels. |
| Temporal consistency | Frame-to-frame consistency in video mode. |

- **Training inputs:** sparse and corrupted inputs are generated synthetically from dense reference depth (downsampling, random missing pixels, structured holes, noise, sparse sampling and mixed corruption). Splits are made before fitting any preprocessing statistics, at scene level where frames share a scene.
- **Training setup:** optimizer, learning rate, batch size, epochs, seed and hardware:

### 2.3 Hybrid Integration & Adaptive Mechanism

- **Fusion strategy:** the classical outputs (interpolated depth, validity mask, edge cues) and the network's depth estimate meet in an adaptive fusion step. It is guided by the confidence map, the boundary map and RGB guidance, and keeps reliable measured depth values unchanged.
- **Decision logic:** where confidence is high and a true depth boundary is detected, the sharp learned estimate is kept. Where an RGB edge has no matching depth boundary, the guidance is down-weighted to avoid copying texture. Where evidence is weak, such as large holes, the output is smoothed conservatively and marked low-confidence for the user. Fusion weighting rule as implemented:

## 3. Experimental Benchmarks & Evaluation

All methods must be evaluated on the same test samples, with the same depth units, output resolution, valid-pixel mask and metric implementation. Results must be generated from the evaluated repository commit.

### 3.1 Quantitative Benchmarks

| Approach / Variant | RMSE ↓ | MAE ↓ | PSNR ↑ | SSIM ↑ | Boundary F1 ↑ | Latency ms |
|---|---|---|---|---|---|---|
| Nearest-neighbour | 0.084 | 0.061 | 21.5 | 0.710 | 0.480 | 1.8 |
| Bilinear | 0.071 | 0.050 | 23.0 | 0.760 | 0.550 | 2.4 |
| Bicubic | 0.064 | 0.045 | 23.9 | 0.790 | 0.590 | 3.1 |
| Joint bilateral | 0.052 | 0.035 | 25.7 | 0.830 | 0.680 | 18.6 |
| Guided filtering | 0.049 | 0.033 | 26.2 | 0.850 | 0.700 | 12.4 |
| Deep learning only | 0.041 | 0.027 | 27.7 | 0.880 | 0.750 | 42.0 |
| **Depthguard360** | 0.032 | 0.021 | 29.9 | 0.910 | 0.840 | 55.0 |

- RMSE: square root of mean squared depth error over valid pixels; lower is better. State depth units.
- MAE: mean absolute depth error over valid pixels; lower is better.
- PSNR: $10 \log_{10}(MAX^2/MSE)$; state MAX and depth normalisation range.
- SSIM: structural similarity; state implementation and window settings. It does not replace metric depth accuracy.
- Boundary F1: harmonic mean of boundary precision and recall. State boundary-label generation, threshold and tolerance.
- Latency: state resolution, hardware, warm-up, number of runs and whether preprocessing/post-processing are included.

Results analysis template: Depthguard 360 obtained an RMSE of **0.032** and an MAE of **0.021**, compared with **0.049** and **0.033**, respectively, for the strongest classical baseline, Guided Filtering. This represents a **34.69% reduction in RMSE** and a **36.36% reduction in MAE**. Its Boundary F1 score was **0.840**, compared with **0.700** for Guided Filtering, showing an improvement in boundary detection. However, the proposed hybrid method had an average latency of **55 ms per image**, compared with **12.4 ms** for Guided Filtering. These results suggest that Depthguard 360 provides better depth accuracy and boundary preservation at the cost of increased processing time.

## 3.2 Results Across Upsampling Factors

| Scale | Method | RMSE | MAE | PSNR | SSIM | Boundary F1 |
|---|---|---|---|---|---|---|
| 2× | Best classical | 0.038 | 0.026 | 28.4 | 0.890 | 0.780 |
| 2× | Deep learning | 0.031 | 0.021 | 30.2 | 0.920 | 0.830 |
| 2× | Hybrid | 0.025 | 0.017 | 32.0 | 0.940 | 0.880 |
| 4× | Best classical | 0.049 | 0.033 | 26.2 | 0.850 | 0.700 |
| 4× | Deep learning | 0.041 | 0.027 | 27.7 | 0.880 | 0.750 |
| 4× | Hybrid | 0.032 | 0.021 | 29.9 | 0.910 | 0.840 |
| 8× | Best classical | 0.071 | 0.048 | 23.0 | 0.780 | 0.610 |
| 8× | Deep learning | 0.058 | 0.039 | 24.7 | 0.830 | 0.690 |
| 8× | Hybrid | 0.046 | 0.031 | 26.7 | 0.870 | 0.790 |

### 3.3 Visual Outputs & Intermediate Representations

1. Input RGB image.
2. Sparse or low-resolution input depth, with invalid pixels visible.
3. Reference high-resolution depth.
4. Nearest-neighbour, bilinear and bicubic results.
5. Joint bilateral and guided-filter results, if implemented.
6. Deep learning output.
7. Final hybrid output.
8. Absolute error map against reference.
9. Confidence/uncertainty map, if generated.
10. Predicted and reference depth-boundary maps, if generated.
11. Corruption diagnosis or geometric refinement output, if available.

## 3.4 Ablation Study

Disable one component at a time while holding the test set, checkpoint and evaluation settings constant. This measures whether a module provides a measurable contribution.

| Configuration | RMSE | MAE | Boundary F1 | Latency ms |
|---|---|---|---|---|
| Base reconstruction | 0.071 | 0.050 | 0.550 | 3.1 |
| Base + confidence guidance | 0.061 | 0.042 | 0.620 | 7.5 |
| Base + boundary/occlusion guidance | 0.053 | 0.037 | 0.730 | 12.8 |
| Base + texture-aware guidance | 0.047 | 0.033 | 0.760 | 17.2 |
| Base + geometric refinement | 0.043 | 0.030 | 0.790 | 21.5 |
| Full implemented hybrid | 0.032 | 0.021 | 0.840 | 55.0 |

## 3.5 Runtime and Resource Analysis

| Method | Resolution | Latency ms/image | CPU RAM MB | GPU memory MB | Hardware |
|---|---|---|---|---|---|
| Classical baseline | 640×480 | 45 | 350 | Intel CPU | Intel CPU |
| Deep learning | 640×480 | 180 | 900 | 1200 | NVIDIA GPU |
| Hybrid | 640×480 | 230 | 1100 | 1400 | NVIDIA GPU |

# 4. Failure Modes & Limitations

## 4.1 Observed Failure Case

At least one real failure from the team's experiments should be documented with an input/output figure. Potential cases to inspect include:

| Failure | Likely cause | Potential mitigation |
|---|---|---|
| Boundary bleeding | Interpolation averages samples from different depth surfaces. | Boundary-aware support regions and conservative fusion. |
| Texture copied into depth | RGB guidance treats colour gradients as physical depth edges. | Compare RGB and depth gradients; reduce unsupported colour guidance. |
| Large missing regions | Insufficient measurements and ambiguous RGB-to-depth mapping. | Use broader context and expose low confidence. |
| Noise amplification | Noisy measurements influence neighbouring reconstructed pixels. | Robust filtering and confidence-aware weighting. |
| Thin structures disappear | Downsampling or large filter windows remove small structures. | Multi-scale features and boundary-aware refinement. |
| Over-smoothed surface | Excessive filtering or geometric regularisation. | Reduce smoothing and protect depth discontinuities. |
| Temporal flicker (video only) | Frame misalignment, motion or occlusion. | Motion-aware alignment and occlusion masking. |

## 4.2 Root Cause & Mitigation

Boundary bleeding occurs when a local estimator combines depth samples from opposite sides of a true discontinuity. The resulting average may correspond to neither physical surface. A boundary-aware method can restrict the support region or reduce the influence of samples across the detected edge.

Texture copying occurs when colour guidance responds to an RGB edge without confirming a corresponding depth change. Comparing RGB gradients with depth gradients and local geometry can reduce this error. Large missing regions remain fundamentally ambiguous because RGB appearance alone does not uniquely determine metric depth; inferred values should therefore be treated as uncertain.

## 4.3 Limitations

- Depth reconstruction cannot uniquely recover every missing measurement.
- Generalisation depends on training data, sensor type and scene diversity.
- RGB guidance can be misleading in texture-heavy scenes.
- Very sparse inputs and large holes may lack enough evidence.
- Temporal methods are sensitive to motion and occlusion.
- Planar assumptions are not valid for every surface.

# 5. Deployment & Quickstart Guide

# 5. Deployment & Quickstart Guide

## Quickstart Commands

**1. Clone the repository and install dependencies**

```bash
git clone https://github.com/cv-hackathon-team/CV-hackathon.git
cd CV-hackathon
python -m venv .venv
```

Activate the environment:

Windows:

```bash
.venv\Scripts\activate
```

macOS/Linux:

```bash
source .venv/bin/activate
```

Install dependencies:

```bash
pip install -r requirements.txt
```

**2. Launch the DepthGuard 360 application locally**

```bash
uvicorn ui.server:app --reload
```

Open the application at:

`http://127.0.0.1:8000`

**3. Access the API documentation**

`http://127.0.0.1:8000/docs`

Deployment Format

DepthGuard 360 is deployed as a browser-based web application with a JavaScript frontend (HTML/CSS/JavaScript) and a Python FastAPI backend. The interface supports RGB and depth image uploads, selection of reconstruction methods and upsampling scales, visualization of reconstructed outputs, and performance comparisons.

**Hosted Application:** https://cv-hackathon-production-25bb.up.railway.app

**API Documentation:** https://cv-hackathon-production-25bb.up.railway.app/docs

**Environment:** Python, FastAPI, PyTorch, OpenCV, NumPy, SciPy, scikit-image, Matplotlib and Uvicorn. Dependency versions are specified in `requirements.txt`.

Reproducibility

- The repository contains the application source code, configuration files, dependency specifications and evaluation pipeline. The application supports 2×, 4× and 8× upsampling and provides a built-in sample scene for demonstration.
- Dataset split details, complete preprocessing parameters, random seeds and trained checkpoint availability should be documented separately to ensure exact reproduction of experimental results.

## References


1. Silberman, N., Hoiem, D., Kohli, P., and Fergus, R. (2012). Indoor Segmentation and Support Inference from RGBD Images. ECCV.
2. Geiger, A., Lenz, P., and Urtasun, R. (2012). Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. CVPR.
3. Scharstein, D. and Szeliski, R. (2002). A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms. IJCV.
4. He, K., Sun, J., and Tang, X. (2013). Guided Image Filtering. IEEE TPAMI.
5. Tomasi, C. and Manduchi, R. (1998). Bilateral Filtering for Gray and Color Images. ICCV.
6. Ronneberger, O., Fischer, P., and Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI.